Programowanie współbieżne w C++

Lista zadań nr 7

Na pracownię 16 kwietnia 2026

Zadanie 1 (4pkty) Optymalizacja porządków w dostępach do pamięci dla kolejki dwuwątkowej i puli elementów.

Wykonaj refaktoryzację swoich implementacji kolejki dwuwątkowej i puli elementów polegającą na optymalizacji porządku dostępów do pamięci w operacjach atomowych. Tam gdzie to możliwe użyj std::memory_order_relaxed. W pozostałych przypadkach wykorzystaj acquire/release. Następnie udowodnij poprawność tak zmodyfikowanego kodu posługując się relacjami sequenced-before, synchronizes-with oraz happens-before.

Zadanie 2 (4pkty) Optymalizacja porządków w dostępach do pamięci dla klasy LockFreeExchanger.

Powtórz poprzednie zadanie dla swojej implementacji klasy LockFreeExchanger.

Zadanie 3 (4pkty) Seqlock — wariant blokady odczytujący–zapisujący.

Zaimplementuj wariant blokady odczytujący–zapisujący (*RW-lock*) przeznaczony dla systemów, w których odczyty współdzielonego zasobu są znacznie częstsze niż zapisy. Mamy jeden wątek zapisujący i dowolnie wiele wątków czytających. Wszystkie wątki współdzielą atomową zmienną całkowitą `seq` (licznik sekwencji), modyfikowaną jedynie przez wątek zapisujący. Wątki czytające jedynie odczytują wartość zmiennej `seq` i na jej podstawie decydują, czy wykonany odczyt był poprawny. Dokładniej:

- Należy stworzyć szablon klasy `template <typename T> class SeqLock`, przechowującej ładunek (payload) typu `T`.
- Klasa będzie posiadać licznik sekwencji: `std::atomic<size_t> seq` o początkowej wartości `0`.
- Pisarz (Writer): 1. Inkrementuje seq na wartość nieparzystą (sygnalizacja rozpoczęcia zapisu). 2. Zapisuje nowe dane do ładunku. 3. Następnie inkrementuje

seq na wartość parzystą by zasygnalizować zakończenie zapisu.

- Czytelnik (Reader): 1. Odczytuje seq (oczekując wartości parzystej). 2. Kopiuje dane do lokalnego bufora. 3. Ponownie odczytuje seq. Jeśli wartość seq zmieniła się od pierwszego odczytu lub jest nieparzysta, czytelnik odrzuca dane i ponawia próbę w pętli.

Zoptymalizuj porządki dostępu do zmiennej atomowej seq. Przetestuj swoją implementację. W jakim scenariuszu Seqlock będzie szczególnie wydajny i dlaczego?

Zadanie 4 (4pkty) Sprawiedliwa blokada (Ticket Lock).

Prosty zamek TAS/TTAS lock omawiany na wykładzie z Programowania współbieżnego nie jest sprawiedliwy, tzn. dopuszcza głodzenie wątków. Celem zadania jest implementacja blokady typu Ticket Lock, działającej na zasadzie pobierania numerków (np. w urzędzie), która gwarantuje zajęcie zamka w kolejności FIFO. Zaimplementuj klasę TicketLock, zawierającą dwa liczniki atomowe: std::atomic<size_t> next_ticket (kolejny wolny numerek) oraz std::atomic<size_t> now_serving (numerek aktualnie obsługiwany). Synchronizacja:

- Wątek w metodzie lock() pobiera numerek inkrementując go jednocześnie przy użyciu next_ticket.fetch_add. Następnie wątek wiruje (spin), oczekując aż now_serving == my_ticket.
- W metodzie unlock() wątek inkrementuje now_serving, umożliwiając zajęcie zamka kolejnemu oczekującemu.

Zoptymalizuj porządki dostępu do zmiennych atomowych i przetestuj swoją implementację. Czy implementacja pozostanie poprawna przy wystąpieniu nadmiaru (overflow) w wartościach zmiennych next_ticket i/lub now_serving, zakładając że liczba oczekujących na zajęcie blokady wątków mieści się w typie size_t? Odpowiedz na to pytanie w komentarzu do zadania. Odpowiedź uzasadnij.